

Engineering Reflection

1. Why should source profiling occur before implementation of ingestion?

Profiling reveals what a source actually contains before any code is written to move it. In this lab, profiling `customers.csv` showed that `customer_id` was not actually unique despite being the intended key, and that some emails and cities were missing. Without profiling first, an ingestion pipeline might have been built on the false assumption that `customer_id` could be trusted as a unique key, which would have caused problems later when that assumption failed.

2. What is the difference between source event time/updated_at and ingestion time?

`_updated_at` is set by the source system and reflects when the underlying record was last changed from the source's point of view. `_ingested_at`, which this pipeline attaches to every API record, reflects when this pipeline actually pulled and processed that record. The two can differ by any amount of time, especially if the pipeline runs late or is delayed. Only `_updated_at` is meaningful for watermark logic, since `_ingested_at` says nothing about the state of the data itself.

3. Why is event_id alone insufficient to decide which duplicate API record to keep in this exercise?

The API intentionally returns some `event_id` values more than once, each time with a different `_updated_at`. If duplicates were resolved by `event_id` alone (for example, keeping whichever copy appeared first), the pipeline could end up keeping a stale version of the record instead of the most recently updated one. The deduplication logic in this pipeline explicitly compares `_updated_at` values across all copies of the same `event_id` and keeps only the one with the greatest timestamp.

4. Why must watermark state advance only after successful persistence?

If the watermark were saved before the raw write completed, a failure during the write (a crash, a disk error) would leave the pipeline believing it had already ingested records that were never actually saved. On the next run, the pipeline would request only records after that watermark, permanently skipping the ones that were lost. In this implementation, `save_watermark()` is only called after the atomic write to `events.jsonl` has fully completed, which was demonstrated directly

in the Task 3.7 failure experiment, where the API was stopped mid-run and the watermark file remained unchanged.

5. What limitation does `updated_after > watermark` have when multiple source records can share exactly the same timestamp?

If several records share the exact same ``updated_at`` value as the current watermark, the strict "greater than" comparison excludes all of them from the next fetch, even if only some of them were actually retrieved in a given run. This could silently drop records at the boundary. This lab's watermark explanation (``config/watermark_explanation.md``) recommends a composite key, such as ``(updated_at, event_id)``, or a strictly increasing sequence number, as a more robust alternative.

6. How is duplicate prevention related to idempotency?

Duplicate prevention is one of the mechanisms that makes a pipeline idempotent. The file ingestion function achieves this by hashing each source file and skipping any file whose exact content has already been ingested. The API ingestion function achieves this by deduplicating on ``event_id`` before writing. Together, these mean that running the pipeline multiple times on the same input never produces duplicate logical records, which was explicitly verified during Task 3.6 by rerunning the pipeline unchanged and then again from a completely clean state.

7. Why should the raw area preserve source values instead of applying business transformations?

Preserving source values in the raw area keeps an unaltered record of exactly what the source provided at the time of ingestion. If a business rule or cleaning step is later found to be wrong, the raw data can still be reprocessed from scratch. If transformations were applied at ingestion time instead, any mistake in that logic would corrupt the only copy of the data, with no way to recover the original values.

8. How could querying a production OLTP source for profiling or extraction degrade the application?

Profiling and extraction queries, especially unbounded ones like full-table scans, large joins, or ``SELECT *`` over an entire table, compete for the same resources (CPU, memory, I/O, locks) that the live application needs to serve real user requests. This lab explicitly required bounded queries

against ``support_tickets``, such as ``LIMIT 10`` and simple ``COUNT(*)`` queries, to avoid placing unnecessary load on the source. A production analytical workload run directly against an OLTP database without such care could slow down or even lock the application it is meant to support.

9. What would you change if the API had a rate limit of 60 requests per minute?

The current pagination loop calls the API as fast as it can, with no delay between requests. With a 60-requests-per-minute limit, the pipeline would need to add a delay of at least one second between page requests, and should also detect and handle HTTP 429 ("Too Many Requests") responses by waiting and retrying rather than failing immediately. For a source with many pages, this could also justify reducing request frequency further or increasing ``per_page`` (up to the API's maximum) to reduce the total number of requests needed.

10. How would you extend this pipeline from a local raw area to PostgreSQL while preserving rerun safety?

The raw files and JSONL output could be loaded into a staging table in PostgreSQL using an upsert pattern (``INSERT ... ON CONFLICT DO UPDATE``) keyed on the same fields already used for deduplication, such as ``event_id``. The file manifest and watermark state could either remain as local JSON files or be moved into their own PostgreSQL tables, as long as the same rule is preserved: the watermark or manifest entry is only updated after the corresponding write to PostgreSQL has been confirmed successful. This preserves the same rerun-safety guarantees demonstrated in this lab, just backed by a database instead of local files.